

```
返回：
- IDCG@k的值。
"""

scores_sorted = np.sort(scores)[::-1]
return dcg_at_k(scores_sorted, k)

def ndcg_at_k(scores, k=5):
    """
    计算NDCG@k。
    参数：
    - scores：一个包含相关性得分的列表或者numpy数组。
    - k：考虑的顶部元素的数量。
    返回：
    - NDCG@k的值。
    """

    dcg = dcg_at_k(scores, k)
    idcg = idcg_at_k(scores, k)
    return dcg / idcg if idcg > 0 else 0

# 示例使用
scores = [3, 2, 3, 0, 1]
k = 3
ndcg_score = ndcg_at_k(scores, k)
print(f"NDCG@{k}: {ndcg_score}")

'''输出值

NDCG@3: 0.9594535145926796
'''
```

1.1.2 生成模块评估

- 生成模块的评估关注于检索到的文档如何增强输入，以形成对查询的响应。与端到端评估中的最终答案/响应生成不同，这里更侧重于评估过程中的上下文相关性。
- 评估指标主要包括BLEU、ROUGE等，这些都是衡量文本生成质量的常用指标。特别是，它们能够评估生成文本的流畅度、一致性以及与原始查询问题的相关性。

(1) BLEU

BLEU (Bilingual Evaluation Understudy) 得分的计算方法，这是一种评估机器翻译质量的指标。它通过比较机器翻译的输出和一组参考翻译来计算得分。

BLEU得分计算步骤

步骤 1: 理解BLEU得分的定义

BLEU得分是一种评价机器翻译输出与一个或多个参考翻译之间相似度的度量。它基于n-gram匹配，计算机器翻译文本和参考翻译文本之间的共有n-gram的数量。得分范围从0到1，接近1意味着更高的翻译质量。

步骤 2: 计算n-gram精确度 P_n

对于每个n-gram，计算机器翻译输出中的该n-gram出现的次数，以及它在参考翻译中出现的最大次数。计算公式为：

$$P_n = \frac{\sum_{i \in \text{n-gram}} \min(h_i(C), \max_{j \in m} h_i(S_j))}{\sum_{i \in \text{n-gram}} h_i(C)} \quad (1)$$

步骤 3: 计算BLEU得分

BLEU得分计算公式如下：

$$\text{BLEU} = \text{BP} \times \exp \left(\sum_{n=1}^N W_n \times \log(P_n) \right) \quad (2)$$

其中， W_n 是对不同n-gram精确度的权重，通常这些权重是相等的。 P_n 是不同n-gram精确度的几何平均值。

步骤 4: 计算BP (Brevity Penalty)

如果机器翻译的输出比参考翻译要短，会通过BP (简短惩罚) 来惩罚得分，以避免过短的输出获得不合理的高分。BP的计算公式如下：

$$\text{BP} = \begin{cases} 1 & \text{if } l_c \geq l_r \\ \exp(1 - l_r/l_c) & \text{if } l_c < l_r \end{cases} \quad (3)$$

这里， l_c 是机器翻译输出的长度，而 l_r 是参考翻译的最佳长度，通常选择与机器翻译输出最接近的参考翻译长度。

通过这些步骤，我们可以计算出机器翻译输出的BLEU得分，从而评估其翻译质量。高的BLEU得分意味着机器翻译与人工翻译的一致性更高。

>>> 举例讲解

通过一个具体的例子来说明BLEU得分的计算过程。

假设我们有一句机器翻译的英语句子和两句参考翻译：

- 机器翻译：the cat is on the mat
- 参考翻译 1：the cat is on the mat
- 参考翻译 2：there is a cat on the mat

步骤 1: 分词 (Tokenization)

首先，我们将所有的句子分成单词 (tokens) 。

步骤 2: n-gram计数

接下来，我们计算n-gram的精确度。为了简化，我们只计算unigram (1-gram) 和bigram (2-gram) 的精确度。

- **Unigram:**

- **机器翻译:** the, cat, is, on, the, mat (每个单词出现1次)
- **参考翻译:** 最多也是每个单词出现1次

因此，unigram的精确度是100%。

- **Bigram:**

- **机器翻译:** the cat, cat is, is on, on the, the mat
- **参考翻译 1:** the cat, cat is, is on, on the, the mat
- **参考翻译 2:** there is, is a, a cat, cat on, on the, the mat

机器翻译中的每个bigram也在至少一个参考翻译中出现过，所以bigram的精确度也是100%。

步骤 3: 计算BLEU得分

接下来，我们计算BLEU得分。为了简化，我们假设所有n-gram的权重相等，即对于unigram和bigram, $W_1 = W_2 = 0.5$ 。

对于unigram和bigram的BLEU计算：

$$\begin{aligned} P_1 &= 1 \\ P_2 &= 1 \end{aligned} \tag{4}$$

BLEU得分为：

$$\begin{aligned} \text{BLEU} &= \text{BP} \times \exp \left(\sum_{n=1}^2 W_n \times \log(P_n) \right) \\ \text{BLEU} &= \text{BP} \times \exp(0.5 \times \log(1) + 0.5 \times \log(1)) \end{aligned} \tag{5}$$

$$\text{BLEU} = \text{BP} \times \exp(0)$$

$$\text{BLEU} = \text{BP} \times 1$$

步骤 4: 计算简短惩罚 (Brevity Penalty, BP)

BP的计算取决于机器翻译输出的长度和参考翻译的最佳长度。在这个例子中，机器翻译的长度和参考翻译的长度相等，因此没有简短惩罚，即BP=1。

最终的BLEU得分是：

$$\text{BLEU} = 1 \times 1 = 1 \tag{6}$$

在这个非常简化的例子中，机器翻译与参考翻译完全相同，因此得分是最高的。实际中，当n-gram越来越长时，匹配会越来越难，因此得分会降低。此外，BLEU通常考虑更长的n-gram（比如4-grams），并使用更复杂的权重分布。

>>> BLEU 计算工具

安装

```
pip install nltk
```

```
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
import jieba

def bleu_score(true_answer : str, generate_answer : str, weights: tuple) -> float:
    '''
        true_answer : 标准答案, str 类型
        generate_answer : 模型生成答案, str 类型
    '''
    # jieba分词
    true_answers = list(jieba.cut(true_answer))
    generate_answers = list(jieba.cut(generate_answer))

    # 使用了平滑函数SmoothingFunction().method4来处理可能的0值情况,
    # 这对于避免由于缺乏高阶n-gram匹配导致的得分为0非常有用。
    smoothie = SmoothingFunction().method4

    # 计算bleu得分
    bleu_score = sentence_bleu(true_answers,
                                generate_answers,
                                weights=weights,
                                smoothing_function=smoothie)

    return bleu_score

# 标准答案
true_answer = "there is a cat on the mat"
# 模型生成答案
generate_answer = "the cat is on the mat"

# 计算BLEU得分
'''权重设置
1-gram BLEU得分：设置weights为(1, 0, 0, 0)
2-gram BLEU得分：设置weights为(0.5, 0.5, 0, 0)，表示1-gram和2-gram各占一半的权重。
3-gram BLEU得分：设置weights为(1/3, 1/3, 1/3, 0)
4-gram BLEU得分：设置weights为(0.25, 0.25, 0.25, 0.25)，这是默认设置，表示1-gram到4-gram均等考
虑。
'''

weights = (0.5, 0.5, 0, 0)
score = bleu_score(true_answer, generate_answer, weights)
print(f"BLEU score: {score}")

'''输出值
```

BLEU score: 0.046689450558483385

...

(2) ROUGE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) 是一种评估自动文摘或机器翻译质量的评价指标，特别是在评估摘要的覆盖度或召回率方面。这里的“召回率”指的是被评估的摘要能在多大程度上覆盖参考摘要中的内容。ROUGE指标与BLEU指标相似，但是**ROUGE更侧重于召回率，而BLEU更侧重于精确度**。

ROUGE的几种变体中，最常见的是ROUGE-N，它计算了参考摘要和生成摘要之间n-gram的重叠情况。计算公式如下：

$$\text{ROUGE-N} = \frac{\sum_{S \in \text{参考摘要}} \sum_{n\text{-gram} \in S} \min(\text{Count}(n\text{-gram}, S), \text{Count}(n\text{-gram}, C))}{\sum_{S \in \text{参考摘要}} \sum_{n\text{-gram} \in S} \text{Count}(n\text{-gram}, S)} \quad (7)$$

其中， C 是机器生成的摘要， S 是参考摘要集合， $\text{Count}(n\text{-gram}, X)$ 是n-gram在文本 X 中的出现次数。

ROUGE-L 是ROUGE的另一个变种，它考虑了最长公共子序列 (LCS)。ROUGE-L的计算不仅仅依赖于n-gram的匹配，而是通过最长公共子序列来衡量整体的序列相似性。其公式如下：

$$R = \frac{\text{LCS}(X, Y)}{|Y|}, \quad P = \frac{\text{LCS}(X, Y)}{|X|} \quad (8)$$

其中， R 是召回率， P 是精确度， $\text{LCS}(X, Y)$ 是参考摘要 Y 和生成摘要 X 的最长公共子序列的长度， $|Y|$ 是参考摘要的长度， $|X|$ 是生成摘要的长度。

>>> 举例讲解

通过一个具体的例子来详细描述ROUGE-N和ROUGE-L的计算过程。

① ROUGE-N 计算过程

假设我们有以下的参考摘要和生成摘要：

- **参考摘要：** The quick brown fox jumps over the lazy dog.
- **生成摘要：** The brown fox jumped over the dog.

为了计算ROUGE-1 (unigram的重叠)，我们首先列出两个摘要中的所有unigrams：

- **参考摘要的Unigrams：** The, quick, brown, fox, jumps, over, the, lazy, dog
- **生成摘要的Unigrams：** The, brown, fox, jumped, over, the, dog

接下来，我们统计生成摘要中每个unigram在参考摘要中出现的次数，并计算总的重叠unigram数：

- The (2次), brown (1次), fox (1次), over (1次), dog (1次) = 6次